

WEEK 0 — ORIENTATION

# AI Open Source Software

---

현대 소프트웨어 개발의 핵심 역량을 습득하는 여정.  
GitHub 중심의 협업, 자동화, 그리고 오픈소스 문화.

## SECTION 01

---

# 환영합니다

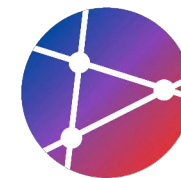
현대 소프트웨어 개발의 핵심 역량을 습득하는 여정에  
오신 것을 환영합니다!

#Welcome #AI\_OSS #StartJourney



# AI OSS 소개

Introduction to AI & Open Source Software



## 과목 범위

인공지능(AI) 기술과 오픈소스 소프트웨어 (OSS) 생태계의 융합을 탐구합니다. 최신 기술 트렌드와 오픈소스의 역할을 이해합니다.



## 학습 방식

GitHub 중심의 협업 워크플로우, GitHub Actions를 활용한 자동화, 그리고 지속적인 문서화 중심의 개발 문화를 익힙니다.



## 실습 중심

이론에 그치지 않고, 개인 및 팀 프로젝트를 통해 실제 배포와 운영까지 경험하며 현대적 소프트웨어 엔지니어링 역량을 체득합니다.



# 이 수업에서 배울 것

## Core Curriculum Overview

student@ai-oss:~/curriculum - zsh

→ `cat course_objectives.txt`

✨ 오픈소스 개발 방법론 # *Open Source Methodology*

🤖 GitHub Actions 자동화 # *Automation*

🚀 CI/CD 파이프라인 구축 # *CI/CD Pipelines*

📊 개발 성과 측정 # *Metrics & Analytics*

👥 협업 도구 마스터 # *Collaboration Tools*

🌐 글로벌 개발 문화 # *Global Dev Culture*

→

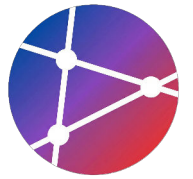
## SECTION 02

# 수업 목표

본 강의의 학습 목표와 성취 기준을 소개합니다. 핵심 역량 4가지를 중심으로 구성됩니다.

#LearningObjectives #CoreCompetencies #DevOps





# Learning Objectives

이번 학기의 주요 학습 목표 및 달성 과제



## 오픈소스 개발 방법론

오픈소스 생태계의 작동 원리를 이해하고, 실제 프로젝트에 기여하며 현대적인 개발 프로세스를 체득합니다.



## CI/CD 자동화 구축

GitHub Actions를 활용하여 빌드, 테스트, 배포 파이프라인을 자동화하고 안정적인 배포 환경을 구성합니다.



## 협업/문서화 개발 문화

Pull Request 기반의 코드 리뷰, 이슈 트래킹, 그리고 철저한 기술 문서화를 통해 팀워크 역량을 강화합니다.



## 품질/테스트 지속적 개선

테스트 커버리지와 코드 품질 메트릭을 기반으로 소프트웨어를 정량적으로 측정하고 지속적으로 개선합니다.

# 핵심 역량 (1/2)

Core Competencies: Modern Practices & DevOps



## 현대적 개발 프랙티스

- ✓ GitHub 기반 협업 워크플로우 숙달
- ✓ 체계적인 Pull Request 작성 및 관리
- ✓ 상호 성장을 위한 코드 리뷰 문화 실천



## 자동화 및 DevOps

- ✓ GitHub Actions 마스터 및 워크플로우 최적화
- ✓ 안정적인 CI/CD 파이프라인 구축
- ✓ 코드로 관리하는 인프라 자동화 (IaC)

# 핵심 역량 (2/2)

Core Competencies: Quality & Collaboration



## 품질 및 테스트

- **테스트 자동화:** 단위 테스트부터 E2E 테스트까지 자동화된 검증 체계 구축
- **코드 품질 관리:** Linter, Formatter 및 정적 분석 도구를 활용한 코드 품질 유지
- **지속적 개선:** 코드 리뷰와 리팩토링을 통한 기술 부채 관리 및 성능 최적화



## 협업 및 커뮤니케이션

- **비동기 협업:** 시공간의 제약을 넘는 효율적인 비동기 커뮤니케이션 방식 습득
- **문서화 중심 개발:** README, Issue, PR 등 텍스트 기반의 명확한 의사소통
- **글로벌 팀워크:** 오픈소스 생태계에서의 글로벌 협업 매너와 규칙 이해



# 강의 구성

16주간 진행될 AI OSS 수업의 전체 커리큘럼을 소개합니다. 기초부터 마스터 과정까지 단계별 학습 로드맵을 확인하세요.

#Curriculum #Roadmap #16Weeks



# 16주 커리큘럼

## Semester Roadmap & Milestones



```
student@ai-oss:~/semester-plan - zsh
```

```
→ tree .
```

```
├── Week 1-4: 기초 (Foundation)
│   ├── Orientation.md
│   ├── Metrics_and_Planning.md
│   ├── Collaborative_Development.md
│   └── Asynchronous_Work.md
├── Week 5-8: 중급 (Intermediate)
│   ├── Open_Source_Contribution.md
│   ├── GitHub_Actions_Basics.yml
│   ├── Workflow_Optimization.md
│   └── Midterm_Presentation.ppt
├── Week 9-12: 고급 (Advanced)
│   ├── Dependency_Management.json
│   ├── Deployment_Automation.sh
│   └── Feature_Flags.config
```

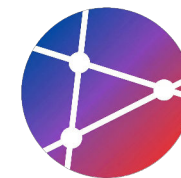
SECTION 04

# 평가 방법

배점 구성 및 평가 포인트 안내

#Grading #Assessment #Criteria





# 배점 구성

Evaluation Criteria & Grading Policy

| 평가 항목                                                                                     | 배점  | 세부 설명                                                   |
|-------------------------------------------------------------------------------------------|-----|---------------------------------------------------------|
|  실습 과제   | 70점 | 주간 과제 (35점) + 개인 프로젝트 (35점)<br>※ 실무 역량 중심의 가장 중요한 평가 요소 |
|  출석/참여   | 10점 | 출석 (5점) + 수업 참여 (5점)                                    |
|  중간 평가   | 10점 | 발표 (5점) + 보고서 (5점)                                      |
|  최종 평가 | 10점 | 발표 (5점) + 회고록 (5점)                                      |

총점 **100점**



# 평가 핵심 포인트

Evaluation Highlights & Key Focus Areas

student@ai-oss:~/evaluation - zsh

→ cat key\_points.md

- ✓ 실습 중심 (70%) # *Practice-oriented*
- ✓ 개인 프로젝트 필수 # *Mandatory Project*
- ✓ GitHub 기반 평가 # *GitHub-based*
- ✓ 지속적 피드백 # *Continuous Feedback*

→ █

## SECTION 05

# 개인 프로젝트

AI OSS 수업의 핵심, 개인 프로젝트의 목표와 산출물,  
운영 기준을 안내합니다.

#Project #Requirements #Evaluation





# 프로젝트 요구사항 (개요)

Project Requirements & Guidelines



## 필수 요소

공개 저장소 운영, 상세 문서화, CI/CD 파이프라인 구축, 테스트 커버리지 달성 등 현대적 개발 환경의 필수 요소를 모두 포함해야 합니다.



## 평가 기준

기획의 명확성부터 코드 품질, 자동화 수준, 테스트 커버리지, 배포 및 운영 상태까지 프로젝트 전반의 완성도를 종합적으로 평가합니다.



## 기술 스택

AI/ML, 웹 애플리케이션, 개발자 도구 등 자유 주제를 선정하되, GitHub Actions와 연동 가능한 현대적 기술 스택을 활용해야 합니다.

# 프로젝트 필수 요소

## Mandatory Requirements Checklist



student@ai-oss:~/project/docs - zsh

→ `cat requirements.md`

1. GitHub Public Repository # 공개 저장소 필수
2. README.md (Detailed Documentation) # 상세 문서화
3. GitHub Actions (CI/CD) # 자동화 파이프라인
4. Tests (Coverage > 80%) # 높은 테스트 커버리지
5. Deployment (Production) # 실제 운영 환경 배포
6. Open Source License # 라이선스 명시 (e.g., MIT)

→



# 프로젝트 평가 기준

Evaluation Criteria & Assessment Points



## 기획 및 설계

5점

프로젝트 목표의 명확성과 시스템 아키텍처의 논리적 설계를 평가합니다.



## 코드 품질

10점

코드의 가독성, 구조적 설계, 그리고 언어별 베스트 프랙티스 준수 여부를 확인합니다.



## 자동화

5점

GitHub Actions를 활용한 CI/CD 파이프라인 구축 및 워크플로우 활용도를 평가합니다.



## 테스트

5점

테스트 코드 작성 여부, 커버리지(80% 이상 권장), 그리고 테스트 품질을 점검합니다.



## 문서화

5점

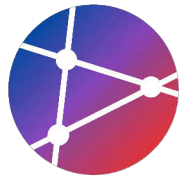
상세한 README, ADR(아키텍처 결정 기록), 기술 문서의 완성도를 평가합니다.



## 배포/운영

5점

실제 운영 환경(Production)으로의 배포 여부와 서비스 동작 상태를 확인합니다.



# 프로젝트 주제 예시

## Overview of Project Topic Categories



### AI/ML 애플리케이션

자연어 처리, 이미지 분석, 추천 시스템 등 인공지능 모델을 활용하여 사용자에게 가치를 제공하는 웹 서비스나 도구를 개발합니다.



### 개발자 도구 (DevTools)

개발자의 생산성을 높이는 CLI 도구, VS Code 확장 프로그램, 코드 생성기, GitHub Actions 등 개발 환경을 개선하는 소프트웨어를 제작합니다.



### 웹 애플리케이션

SaaS 프로덕트, 데이터 시각화 대시보드, 커뮤니티 플랫폼, API 서버 등 실제 운영 환경을 고려한 완성도 높은 웹 서비스를 구축합니다.



### 오픈소스 라이브러리

다른 개발자들이 프로젝트에서 활용할 수 있는 유틸리티 라이브러리, 프레임워크 플러그인, SDK 또는 API Wrapper를 설계하고 배포합니다.

# 프로젝트 주제 예시 (1/2)

## Project Topic Examples: Part 1



student@ai-oss:~/ideas - zsh

→ `cat 01_ai_ml_apps.md`

### ### AI/ML Applications

- 자연어 처리 웹 서비스 # *NLP Web Service*
- 이미지 분류/생성 도구 # *CV Tools*
- 추천 시스템 # *Recommendation Engine*
- 챗봇 플랫폼 # *Conversational AI*

→ `cat 02_developer_tools.md`

### ### Developer Tools

- CLI 도구 # *Command Line Interface*
- VS Code Extension # *Editor Plugins*



# 프로젝트 주제 예시 (2/2)

Project Examples: Web Apps & Libraries

student@ai-oss:~/ideas - zsh

→ `cat web_applications.yml`

🌐 SaaS 프로젝트 # *Software as a Service*

📊 대시보드 # *Data Visualization & Admin*

🔌 API 서버 # *Backend REST/GraphQL Services*

👥 커뮤니티 플랫폼 # *Social & Forum Systems*

→ `cat open_source_libs.yml`

📦 유틸리티 라이브러리 # *Common Helper Functions*

🧩 프레임워크 플러그인 # *React/Vue/FastAPI Extensions*

🔧 SDK/API Wrapper # *Client Libraries for Public APIs*

SECTION 07

# 개발 환경 설정

필수 도구 및 환경 점검

#Environment\_Setup #Git #IDE #Terminal



# 필수 도구

Essential Tools for Development Environment



## Git & GitHub

버전 관리와 협업을 위한 필수 플랫폼. 소스 코드 이력을 관리하고 팀원과 코드를 공유합니다.



## 에디터/IDE

효율적인 코드 작성을 위한 도구. VS Code, IntelliJ 등 생산성을 높여주는 환경을 구축합니다.



## 터미널

CLI(Command Line Interface) 환경에 익숙해져야 합니다. 시스템 제어와 자동화의 핵심입니다.



## 언어/런타임

프로젝트 실행을 위한 기반 환경. Python, Node.js, Java 등 필요한 언어 팩을 설치합니다.



# 1) Git & GitHub

## Essential Setup & Configuration

```
student@ai-oss:~/setup - bash

# 1. Git 설치 확인
$ git --version
git version 2.43.0

# 2. 사용자 정보 설정 (커밋 기록용)
$ git config --global user.name "Sejin Chun"
$ git config --global user.email "sejin.chun@donga.ac.kr"

# 3. GitHub 연동을 위한 SSH 키 생성
$ ssh-keygen -t ed25519 -C "sejin.chun@donga.ac.kr"
Generating public/private ed25519 key pair...

# 4. 생성된 공개키 확인 (이 값을 GitHub에 등록)
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... (copy this)
```

# 에디터 및 IDE

## Development Environment Setup



student@ai-oss:~/tools - zsh

→ `cat recommended_editors.list`

★ **Visual Studio Code (VS Code)** *// Highly Recommended!*

- IntelliJ IDEA
- Vim / Neovim
- Sublime Text

→ `cat vscode_extensions.req`

📦 **GitLens** *# Git supercharged*

🐙 **GitHub Pull Requests** *# PR management*

▶ **GitHub Actions** *# CI/CD workflows*

✨ **Prettier** *# Code formatter*

🔍 **ESLint** *# JavaScript linting*

→



# 개발 환경 설정: 터미널

Essential Tools - Terminal & Shell



student@ai-oss:~/setup - zsh

→ `cat recommended_terminals.txt`

## Windows Environment

- ✓ Windows Terminal # *Highly Recommended (Microsoft Store)*
- PowerShell 7+ # *Modern Cross-platform Shell*
- Git Bash # *Unix-like Emulation*

## macOS / Linux

- ✓ iTerm2 # *macOS Standard (Better than default)*
- Zsh + Oh My Zsh # *Powerful Shell Enhancement*
- Starship # *Fast, Customizable Prompt*

# 언어 및 런타임 선택

Programming Languages & Runtimes



student@ai-oss:~/setup - zsh

```
→ cat supported_versions.conf
```

🟢 Node.js 18+ # *JavaScript / TypeScript*

🟡 Python 3.10+ # *AI & Data Science*

🐼 Go 1.20+ # *Backend Services*

☕ Java 17+ # *Enterprise Apps*

🦀 Rust 1.70+ # *Systems Programming*

```
→ echo "Select one for your project"
```

# 개발 환경 체크리스트

Final Environment Verification



student@ai-oss:~/setup - zsh

→ cat checklist.md

#  체크리스트

- [ ] Git 설치 및 설정
- [ ] GitHub 계정 생성
- [ ] SSH 키 등록
- [ ] 에디터/IDE 설치 (VS Code 권장)
- [ ] 필수 확장 프로그램 설치
- [ ] 언어/런타임 설치 (Node.js/Python 등)
- [ ] 터미널 설정
- [ ] 테스트 레포지토리 생성

# GitHub 기초

Repository 생성부터 기본 워크플로우까지, 협업을 위한 GitHub의 핵심 기능을 학습합니다.

[#GitHub](#) [#Collaboration](#) [#VersionControl](#)



# GitHub Repository 생성

## Initial Setup & Basic Workflow



student@ai-oss:~/projects - zsh

```
# 1. GitHub 웹사이트에서 새 레포지토리(New Repository) 생성
# Settings: Public, Add README, Add .gitignore (Python/Node), MIT License

# 2. 로컬 개발 환경으로 클론
→ git clone git@github.com:username/ai-oss-project.git
→ cd ai-oss-project

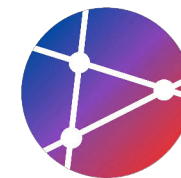
# 3. 기능 개발을 위한 브랜치 생성
→ git checkout -b feature/initial-setup
   Switched to a new branch 'feature/initial-setup'

# 4. 변경사항 스테이징 및 커밋
→ git add .
→ git commit -m "feat: initial project setup"
   [feature/initial-setup 1a2b3c] feat: initial project setup

# 5. 원격 저장소로 푸시
→ git push origin feature/initial-setup
```

# README 작성

## Standard Project Documentation Guide



README.md

Markdown

### # Project Name

간단한 프로젝트 설명 (Description)

### ## 🎯 Features

- 주요 기능 1 // Key Feature 1
- 주요 기능 2 // Key Feature 2
- 주요 기능 3 // Key Feature 3

### ## 🚀 Getting Started

#### ### Prerequisites

- Node.js 18+
- npm or yarn

#### ### Installation

```
git clone https://github.com/user/project.git
cd project
npm install
```

#### ### Usage

```
npm start
```

### ## 🧪 Testing

```
npm test
```

### ## 📄 License

MIT License

## SECTION 08

# 주차별 일정

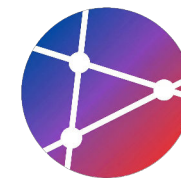
16주간 진행될 강의의 전체 로드맵과 주요 마일스톤을 확인합니다.

#Curriculum #Roadmap #16Weeks



# 16주 전체 일정

Weekly Schedule & Assignments



| Week | 주제                       | 과제 / 활동            |
|------|--------------------------|--------------------|
| 1    | Orientation              | 프로젝트 주제 제안         |
| 2    | Metrics That Matter      | 메트릭 대시보드 설계        |
| 3    | Plan, Track, Visualize   | GitHub Projects 설정 |
| 4    | Teamwork & Collaboration | PR 워크플로우 구축        |
| 5    | Asynchronous Work        | 문서화 (ADR)          |
| 6    | Open & Inner Source      | 오픈소스 기여            |
| 7    | GitHub Actions Basics    | CI 워크플로우           |
| 8    | Running Workflows        | 중간 발표              |
| 9    | Dependency Management    | Dependabot 설정      |
| 10   | Deploying to Platform    | 배포 자동화             |
| 11   | Feature Flags            | 기능 플래그 구현          |



## SECTION 09

# 학습 전략

수업 참여 방식과 자기 주도 학습 팁. 성공적인 학습을 위한 4가지 핵심 전략을 소개합니다.

#LearningStrategy #ActiveParticipation #Growth





# 전략 1: 적극적 참여

## Active Participation

student@ai-oss:~/strategies - zsh

→ `cat 01_active_participation.md`

✓ 질문하기 (망설이지 말 것!) *# Don't hesitate to ask*

✓ 토론 참여 *# Join discussions*

✓ GitHub Discussions 활용 *# Use platform features*

✓ 팀원과 협업 *# Collaborate with team*

→ █



## 전략 2: 꾸준한 실습

Learning Strategy 02 - Consistency is Key

student@ai-oss:~/strategies - zsh

→ `check_habits --daily`

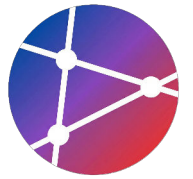
✓ 매주 과제 제출 # *Weekly Submission*

✓ 조금씩 자주 커밋 # *Atomic Commits*

✓ 다양한 도구 실험 # *Experimentation*

✓ 실패를 두려워하지 말 것 # *Fail Fast, Learn Faster*

→ █



## 전략 3: 문서화 습관

### Documentation First Approach

student@ai-oss:~/strategies - zsh

→ `cat 03_documentation_habit.md`

✓ **README 작성** # 상세한 프로젝트 대문 만들기

✓ **코드 주석** # "Why"에 집중하여 작성

✓ **커밋 메시지** # *Conventional Commits* 준수

✓ **학습 일지** # *TIL (Today I Learned)* 기록

→ `echo "Code tells what, Docs tell why"`



## 전략 4: 커뮤니티 활용

Community & Knowledge Sharing

```
student@ai-oss:~/learning-strategies - zsh
```

```
→ cat community_resources.list
```

```
✓ Stack Overflow    # Global Q&A Knowledge Base
```

```
✓ GitHub Discussions # Project-specific Support
```

```
✓ Discord / Slack   # Real-time Communication
```

```
✓ 기술 블로그 (Tech Blogs) # Deep Dive & Trends
```

```
→ echo "Learn together, grow faster!"
```

```
Learn together, grow faster!
```

## SECTION 11

# 추천 학습 자료

필수 문서, 온라인 강의, 도서, 팟캐스트 등 학습에 도움  
되는 리소스 모음

[#Learning\\_Resources](#) [#Books](#) [#Docs](#)



# 추천 학습 자료

Curated Essential Resources for AI OSS



## 필수 읽기 (Must Read)

- > **GitHub Docs:** 공식 문서, 기능별 상세 가이드
- > **GitHub Skills:** 인터랙티브 학습 코스
- > **Pro Git:** Git의 모든 것을 다루는 필독서 (무료)



## 온라인 강의 (Courses)

- > **GitHub Learning Lab:** 실습 중심의 자동화 봇 학습
- > **freeCodeCamp:** Git & GitHub Crash Course
- > **Udacity:** Version Control with Git (무료 강좌)



## 도서 (Books)

- > **Accelerate:** DevOps 성과 측정과 조직 문화
- > **The DevOps Handbook:** 기술 부채와 배포 파이프라인
- > **Working in Public:** 오픈소스 유지보수의 현실
- > **Clean Code:** 지속 가능한 코드를 위한 지침서



## 팟캐스트 (Podcasts)

- > **Software Engineering Daily:** 기술 트렌드 심층 인터뷰
- > **The Changelog:** 오픈소스 개발자들의 이야기
- > **Developer Tea:** 개발자의 성장과 마인드셋

## SECTION 10

# 소통 채널

수업 중 사용되는 주요 커뮤니케이션 채널과 활용 방법을 안내합니다.

#Communication #GitHub #OfficeHour





# 소통 채널

Communication: GitHub, Email & Office Hours



```
student@ai-oss:~/contact - zsh
```

```
→ cat channels.conf
```

```
platform "GitHub" {  
    # Primary Communication Platform  
    Issues = "기술적 질문 (Technical Q&A)"  
    Discussions = "일반 토론 (General Topics)"  
    PRs = "코드 리뷰 (Code Review)"  
    Projects = "작업 추적 (Task Tracking)"  
}  
  
contact "Email" {  
    # Personal Inquiries Only  
    To = "[교수 이메일]"  
    Response = "24시간 이내 (Within 24h)"  
}
```

## SECTION 12

---

# 주의사항

학습 윤리, 제출 규칙, 출석 정책 등 성공적인 과정 이수를 위해 반드시 지켜야 할 사항들을 안내합니다.

#Ethics #Policy #Rules





# 학습 윤리: 허용 vs 금지

## Academic Integrity Guidelines

student@ai-oss:~/ethics - zsh

→ cat ALLOWED.md

- ✓ AI 도구 사용 (*GitHub Copilot, ChatGPT*)
  - └─ 단, 코드의 동작 원리를 완전히 이해하고 사용할 것
- ✓ 오픈소스 코드 참고
  - └─ 단, 출처 명시 및 라이선스 준수 필수
- ✓ 온라인 자료 참고 & 팀원과 토론

→ cat FORBIDDEN.md

- ✗ 코드 표절 # *Plagiarism*
- ✗ 타인 작업물 복사 # *Copying*
- ✗ 과제 대리 제출 # *Cheating*
- ✗ AI 생성 코드 무분별 사용 # *Blind Copy-Paste*



# 제출 규칙 및 출석 정책

## Submission Rules & Attendance Policy

student@ai-oss:~/course-info - zsh

→ `cat submission_rules.txt`



제출 기한: 매주 일요일 23:59



제출 방법: GitHub Repository (*Pull Request & Issue*)



늦은 제출: 1일당 10% 감점 (최대 3일 허용)

→ `cat attendance_policy.conf`



4회 이상 결석: F 학점 처리



지각 2회 = 결석 1회



공결 처리: 사전/사후 증빙 서류 제출 필수

→ `echo "Be Punctual & Responsible!"`

## SECTION 14

# 첫 주 과제

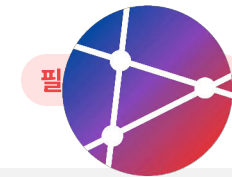
개발 환경 설정, 프로젝트 주제 제안, 자기소개 등 이번 주차에 완료해야 할 필수 과제들을 안내합니다.

[#Assignments](#) [#Setup](#) [#Proposal](#) [#StartNow](#)



# 과제 1: 환경 설정

## Assignment 01: Environment Setup



### 개발 환경 구축 체크리스트

다음 6가지 항목을 모두 완료하고 인증하세요.



#### Git 설치 및 설정

로컬 설치 후 `git config` 로 사용자 이름/이메일 설정 완료



#### GitHub 계정 생성

본인 식별 가능한 프로필 사진 및 이름 설정 권장



#### SSH 키 등록

로컬 터미널과 GitHub 간 안전한 통신을 위해 SSH 키 생성 및 등록



#### 테스트 레포지토리 생성

Public 설정으로 `practice-repo` 생성



#### README.md 작성

Markdown 문법을 사용하여 간단한 자기소개 또는 프로젝트 설명 작성



#### 첫 커밋 & 푸시

로컬 변경사항을 원격 저장소에 반영 (`git push`)



기한: 1주차 일요일 23:59

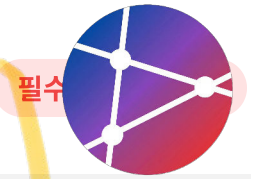


제출: GitHub Issue 댓글로 레포지토리 링크 공유

난이도: ★☆☆ (하)

# 과제 2: 프로젝트 주제 제안서

Assignment 02: Project Proposal



## 제안서 작성 가이드

프로젝트의 방향성을 결정하는 중요한 문서입니다. 구체적으로 작성해주세요.



### 기본 정보 (Basic Info)

**프로젝트명:** 직관적이고 기억하기 쉬운 이름

**개요:** 프로젝트를 한 문장으로 설명

**동기:** 왜 이 프로젝트를 시작하는지? (Why)

**예상 결과물:** 최종적으로 완성될 형태



### 기술 스택 (Tech Stack)

**Frontend:** React, Vue, Svelte 등

**Backend:** Node.js, Python, Go 등

**Database:** PostgreSQL, MongoDB 등

**Deployment:** Vercel, AWS, Cloudflare 등



### 주요 기능 (Key Features)

- 핵심 기능 1 (Core Feature)
- 핵심 기능 2 (Core Feature)
- 핵심 기능 3 (Core Feature)



### 마일스톤 (Milestones)

**W 1-4:** 기획, 설계, 초기 세팅

**W 5-8:** 핵심 기능 구현, MVP

**W 9-12:** 고도화, 추가 기능, 테스트

**W 13-16:** 최적화, 문서화, 최종 발표



기한: 1주차 일요일 23:59

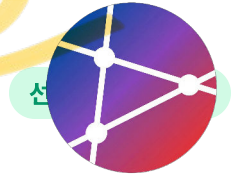


제출: GitHub Issue (Label: `project-proposal`)

난이도: ★★★ (중)

# 과제 3: 자기소개

Assignment 03: GitHub Profile README



## GitHub Profile README 작성

자신을 표현하는 매력적인 프로필 페이지를 만들어보세요.

### 포함되어야 할 내용



본인 소개 (Who am I)



관심 분야 (Interests)



기술 스택 (Tech Stack)



학습 목표 (Learning Goals)



연락처 (Contact Info)

### 👁 Markdown 예시

### 👋 Hi there, I'm Sejin!

- 🌱 I'm currently learning **\*\*Open Source Software\*\***
- 👥 I'm looking to collaborate on **\*\*AI Projects\*\***
- 😊 Ask me about **\*\*Python, Machine Learning\*\***
- 📧 How to reach me: **\*\*email@example.com\*\***

#### 🌟 Tech Stack

![Python](https://img.shields.io/badge/Python-3776AB?

style=flat&logo=python&logoColor=white)

![Git](https://img.shields.io/badge/Git-F05032?style=flat&logo=git&logoColor=white)



기한: 자율 제출 (학기 중 언제든지)



생성 방법: 본인 계정명과 동일한 이름의 Repository 생성 (Public)

보너스 점수: 없음 (네트워킹용)



## SECTION 14

# 시작하기

오늘 바로 실행할 액션 아이템을 확인하고  
본격적인 개발 환경을 준비합니다.

[#ActionableItems](#) [#GitSetup](#) [#StartNow](#)





# 지금 바로 할 일

Action Items: Setup & Configuration

```
student@home:~/setup - bash

→ cat setup_checklist.sh

# 1. GitHub 계정 생성 (없다면)
open https://github.com/signup

# 2. Git 설치
# Windows: https://git-scm.com/download/win
# macOS: brew install git
# Linux: sudo apt-get install git

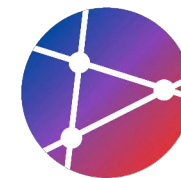
# 3. Git 설정
git config --global user.name "Your Name"
git config --global user.email "your@email.com"

# 4. SSH 키 생성 & 등록
ssh-keygen -t ed25519 -C "your@email.com"
cat ~/.ssh/id_ed25519.pub # GitHub Settings에 복사

# 5. 연결 테스트 & 첫 레포지토리
```

# 핵심 요약 & 마지막 조언

Key Takeaways & Tips for Success



## Process & Culture

- ✓ **실습 중심 (Hands-on):** 이론보다 실제 구현을 통해 배웁니다.
- ✓ **GitHub 협업:** PR, 코드 리뷰 등 협업 워크플로우를 체화합니다.
- ✓ **개인 프로젝트:** 기획부터 배포까지 주도적으로 수행합니다.



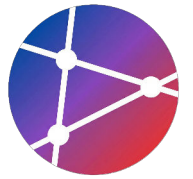
## Technical Skills

- 🔧 **자동화 (CI/CD):** GitHub Actions로 개발 파이프라인을 구축합니다.
- 🔪 **품질 관리:** 테스트 커버리지와 코드 품질을 유지합니다.
- 📄 **문서화:** README, 이슈, 커밋 메시지로 소통합니다.



## Success Tips

- ★ **Start Early:** 과제를 미루지 말고 일찍 시작하세요.
- ★ **Commit Often:** 작게, 자주 커밋하여 과정을 기록하세요.
- ★ **Ask & Share:** 적극적으로 질문하고 지식을 공유하세요.



# 유용한 링크 및 연락처

Resources & Contact Information



## 공식 문서 및 학습 자료



GitHub Docs / Actions Reference



Git Documentation (Pro Git)



GitHub Skills & Learning Lab



Learn Git Branching (Interactive)



## 커뮤니티



GitHub Community



Stack Overflow



Dev.to



## 연락처 및 채널

### 교수진



**Prof. Sejin Chun**

Dong-A University



[교수 이메일]



[연구실 위치] (오피스 아워: [시간])

### 수업 채널



GitHub Organization: [링크]



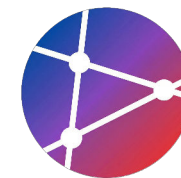
Discord/Slack: [초대 링크]



LMS: [링크]

# 환영합니다!

Welcome & Next Steps



OSS Course 2025  
Let's Build Something Amazing Together! 🚀

여러분의 오픈소스 개발 여정을  
진심으로 응원합니다!

*"The best way to predict the future is to invent it."*

— Alan Kay

## NEXT WEEK PREVIEW



Week 2 Topic

### Metrics That Matter

데이터 기반의 의사결정과 개발 생산성 측정

지금 바로 시작하세요! 🚀